

Trie

A Problem-Solving Companion to Striver's A2Z DSA Sheet

Maruri Saikiran

June 29, 2026

Contents

1	Trie	1
1.1	What Problem Does a Trie Actually Solve?	1
1.2	Implement Trie (Insert, Search, StartsWith)	3
1.3	Implement Trie II (Count and Erase)	5
1.4	Longest String with All Prefixes	8
1.5	Number of Distinct Substrings in a String	11
1.6	Maximum XOR of Two Numbers in an Array	13
1.7	Maximum XOR With an Element from Array (Queries)	16

Chapter 1

Trie

A trie trades memory for a kind of search no comparison-based structure can offer: it does not ask “is this string less than or greater than that one” – it asks “what character comes next,” one position at a time, branching only where words actually disagree.

1.1 What Problem Does a Trie Actually Solve?

A **trie** (prefix tree) stores a collection of strings (or, viewed differently, a collection of fixed-length binary numbers) as paths from a shared root, where every node represents one character (or one bit) consumed so far, and nodes are shared across all strings that agree on a prefix. Two strings that share a k -character prefix share the first k nodes of their paths through the trie; they only diverge into separate branches at the first character where they differ.

Why This Beats a Hash Set or a BST for Prefix Questions

A hash set answers “is this exact string present” in $O(\text{length})$, but cannot answer “how many stored strings start with this prefix” without scanning every stored string. A BST of strings can answer prefix questions via range queries, but each comparison still costs $O(\text{length})$ in the worst case, and the tree’s shape depends on insertion order. A trie answers **both** exact-match and prefix-match queries in $O(\text{length of the query})$, completely independent of how many strings are stored or in what order they were inserted – because shared prefixes are physically shared as the same path, prefix counting is just a matter of annotating nodes along that shared path with counts as strings are inserted.

How to Recognise a Trie Problem from Its Wording

Recurring Trie Keywords

- **“Insert/search words,” “starts with,” “prefix”** – a direct, textbook trie: one node per character, a marker at nodes where a complete word ends.
- **“Count words equal to / starting with a given string,” or “erase a word”** – the same trie, with each node additionally annotated with a small counter (how many inserted words pass through, or end at, this node), updated incrementally on insert/erase.
- **“Longest prefix that is itself always a complete, previously-seen word” or “every prefix of this word must also be a complete word”** – a trie where the search/build step explicitly checks, at every node along a path, whether that node marks the end of *some* inserted word, not just whether the final node does.
- **“Maximum XOR” between numbers, or “maximum XOR with an element from an array,” for a query** – a **binary trie**: treat each number as a fixed-length sequence of bits (e.g. 32 bits), insert each number bit by bit from the most significant bit downward, and answer a maximum-XOR query by greedily choosing, at each bit

position, the branch that *disagrees* with the query number's corresponding bit (since XOR-ing differing bits produces a 1, maximizing the result).

- **“Number of distinct substrings”** – build a trie of *every suffix* of the string; the number of distinct substrings equals the number of nodes created during this construction (every trie node corresponds to exactly one distinct substring, the one spelled out by the path from the root to that node).

Each section in this chapter follows the established structure: problem statement and keywords; the reasoning that identifies the trie variant needed and why it is correct; the algorithm in words and pseudocode; a hand-traced dry run; complete compilable C++ code; and a time/space complexity analysis.

1.2 Implement Trie (Insert, Search, StartsWith)

Problem Statement. Implement a trie supporting *insert(word)*, *search(word)* (returns true only if *word* was previously inserted as a *complete* word), and *startsWith(prefix)* (returns true if any inserted word begins with *prefix*).

Pattern Recognition

This is the foundational trie, with no special twist – the rest of this chapter builds directly on the node structure and walk logic derived here. The one subtlety worth flagging immediately: *search* and *startsWith* use the *same* walk, differing only in what they check once the walk finishes.

Understanding the problem carefully

Each trie node represents one character position along some prefix, and holds up to 26 child pointers (one per lowercase letter, say), plus a boolean flag marking whether a complete inserted word *ends* exactly at this node. Walking a string through the trie, one character at a time, either follows existing child pointers (if a previous word shares this prefix) or creates new nodes (during insertion, for the first word to diverge here).

One Walk, Three Different Endings

insert(word): starting at the root, for each character, create the corresponding child if it does not yet exist, then move into it; after the last character, mark the final node's end-of-word flag. *search(word)*: walk the same way, but if any required child is missing, return false immediately; after the last character, return whether the final node's end-of-word flag is set (not just whether the path exists – *word* might only be a *prefix* of something inserted, not itself a complete inserted word). *startsWith(prefix)*: identical walk, but return true as soon as the path exists in full, regardless of the end-of-word flag.

Why search must check the end-of-word flag but startsWith must not

This is the one place a careless implementation typically goes wrong: if only “apple” has been inserted, then *search("app")* must return **false** (“app” was never inserted as a complete word, even though it is a valid path in the trie), while *startsWith("app")* must return **true** (something inserted does begin with “app”). The trie's path existing is necessary but not sufficient for *search* – it additionally needs the explicit end-of-word marker, which is exactly the piece of information *startsWith* does not need to check at all.

Algorithm

1. *INSERT(word)*: *node* $\leftarrow$ *root*. For each character *c* in *word*: if *node* has no child *c*, create one; move *node* into child *c*. Mark *node.isEnd* $\leftarrow$ true.
2. *SEARCH(word)*: walk as above; if any required child is missing, return false; otherwise return *node.isEnd*.
3. *STARTSWITH(prefix)*: walk as above; return true if the full walk succeeds (regardless of *isEnd*), false if any child is missing.

Dry run

Dry Run. Insert "apple". Query *search("app")*, *startsWith("app")*, *search("apple")*.

- Insert "apple": creates a path of 5 nodes, $a \rightarrow p \rightarrow p \rightarrow l \rightarrow e$, marking only the final (e) node's *isEnd* flag.
- *search*("app"): path $a \rightarrow p \rightarrow p$ exists, but that node's *isEnd* is false (only the e node at the end of "apple" has it set). Returns **false**.
- *startsWith*("app"): path exists, so returns **true**, regardless of the flag.
- *search*("apple"): path exists and the final node's *isEnd* is true. Returns **true**.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TrieNode {
5     TrieNode* children[26] = {nullptr};
6     bool isEnd = false;
7 };
8
9 class Trie {
10     TrieNode* root;
11
12 public:
13     Trie() { root = new TrieNode(); }
14
15     void insert(string word) {
16         TrieNode* node = root;
17         for (char c : word) {
18             int idx = c - 'a';
19             if (!node->children[idx]) node->children[idx] = new
20                 TrieNode();
21             node = node->children[idx];
22         }
23         node->isEnd = true;
24     }
25
26     bool search(string word) {
27         TrieNode* node = root;
28         for (char c : word) {
29             int idx = c - 'a';
30             if (!node->children[idx]) return false;
31             node = node->children[idx];
32         }
33         return node->isEnd;
34     }
35
36     bool startsWith(string prefix) {
37         TrieNode* node = root;
38         for (char c : prefix) {
39             int idx = c - 'a';
40             if (!node->children[idx]) return false;
41             node = node->children[idx];
42         }
43         return true;
44     }
45 };

```

```

46 int main() {
47     Trie trie;
48     trie.insert("apple");
49     cout << trie.search("app") << endl;           // 0 (false)
50     cout << trie.startsWith("app") << endl;       // 1 (true)
51     cout << trie.search("apple") << endl;         // 1 (true)
52     return 0;
53 }

```

Complexity Analysis

Time Complexity. $O(L)$ for each operation, where L is the length of the word/prefix involved – independent of how many other words are stored in the trie. **Space Complexity.** $O(\Sigma \cdot N)$ in the worst case, where N is the total number of characters inserted across all words and Σ is the alphabet size (each node reserves space for all 26 possible children, whether used or not); shared prefixes reduce this in practice.

Key Takeaway

The entire chapter’s machinery is this one node structure (a fixed-size child array plus an end marker) and this one walk (follow or create children, one character at a time). Every later section either adds more annotations to each node (counts, in Section 1.3) or changes what “character” means (bits, in Sections 1.6–1.7).

1.3 Implement Trie II (Count and Erase)

Problem Statement. Implement a trie supporting *insert(word)*, *countWordsEqualTo(word)* (how many times *word* has been inserted, counting duplicates), *countWordsStartingWith(prefix)* (how many inserted words, counting duplicates, begin with *prefix*), and *erase(word)* (remove one occurrence of a previously inserted *word*).

Pattern Recognition

The keyword shift from Section 1.2 is from a yes/no question (*is this word present*) to a **counting** question (*how many times*, including duplicates and including prefix counts), plus **removal**. The fix is to annotate every node, not just mark a single boolean: track how many inserted words **pass through** this node (as part of some longer word, or ending exactly here), and how many **end exactly** at this node.

Understanding the problem carefully

Every node along the path of an inserted word is, by definition, a prefix of that word – so if we increment a *countPrefix* counter at *every* node visited during an insertion, that counter at any given node ends up holding exactly the number of inserted words (counting duplicates) that share this far as a common prefix. Separately, incrementing a *countEnd* counter only at the *final* node of each inserted word tracks exactly how many times that complete word itself was inserted.

Two Counters per Node: Prefix-Passes and Exact-Ends

insert(word): walk the word’s path, creating nodes as needed (as in Section 1.2), and increment *countPrefix* at *every* node visited along the way (including the final one); increment *countEnd* only at the final node. *countWordsEqualTo(word)*: walk the path; if it exists in

full, return the final node's *countEnd* (else 0). *countWordsStartingWith(prefix)*: walk the path; if it exists in full, return the final node's *countPrefix* (else 0). *erase(word)*: walk the path (assuming *word* was indeed previously inserted), **decrementing** *countPrefix* at every node visited, and decrementing *countEnd* at the final node.

Why two separate counters are needed, not just one

A single counter cannot distinguish “*prefix* is itself a complete inserted word *k* times” from “*prefix* is merely a prefix of *k* longer inserted words.” For example, after inserting “app” once and “apple” once, the node at the end of “app” has *countPrefix* = 2 (both words pass through it) but *countEnd* = 1 (only “app” itself ends there) – *countWordsEqualTo*(“apple”) must report 1, while *countWordsStartingWith*(“app”) must report 2, and no single counter can serve both queries correctly.

Algorithm

1. *INSERT(word)*: walk/create the path; increment *countPrefix* at every visited node; increment *countEnd* at the final node.
2. *COUNTWORDSEQUALTO(word)*: walk the path; return the final node's *countEnd*, or 0 if the path does not fully exist.
3. *COUNTWORDSTARTINGWITH(prefix)*: walk the path; return the final node's *countPrefix*, or 0 if the path does not fully exist.
4. *ERASE(word)*: walk the path; decrement *countPrefix* at every visited node; decrement *countEnd* at the final node.

Dry run

Dry Run. Insert “apple” twice, insert “app” once. Query *countWordsEqualTo*(“apple”), *countWordsStartingWith*(“app”). Then erase “apple” once and repeat both queries.

- After both insertions: node at “app” has *countPrefix* = 3 (all three insertions pass through it), *countEnd* = 1 (only the single “app” insertion ends there). Node at “apple” has *countPrefix* = 2, *countEnd* = 2.
- *countWordsEqualTo*(“apple”) = 2. *countWordsStartingWith*(“app”) = 3.
- Erase “apple” once: decrement *countPrefix* along the whole path (node at “app”: 3 → 2; node at “apple”: 2 → 1), and *countEnd* at the final node (2 → 1).
- Now *countWordsEqualTo*(“apple”) = 1. *countWordsStartingWith*(“app”) = 2.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TrieNode {
5     TrieNode* children[26] = {nullptr};
6     int countPrefix = 0;
7     int countEnd = 0;
8 };

```

```

9
10 class TrieII {
11     TrieNode* root;
12
13 public:
14     TrieII() { root = new TrieNode(); }
15
16     void insert(string word) {
17         TrieNode* node = root;
18         for (char c : word) {
19             int idx = c - 'a';
20             if (!node->children[idx]) node->children[idx] = new
                TrieNode();
21             node = node->children[idx];
22             node->countPrefix++;
23         }
24         node->countEnd++;
25     }
26
27     int countWordsEqualTo(string word) {
28         TrieNode* node = root;
29         for (char c : word) {
30             int idx = c - 'a';
31             if (!node->children[idx]) return 0;
32             node = node->children[idx];
33         }
34         return node->countEnd;
35     }
36
37     int countWordsStartingWith(string prefix) {
38         TrieNode* node = root;
39         for (char c : prefix) {
40             int idx = c - 'a';
41             if (!node->children[idx]) return 0;
42             node = node->children[idx];
43         }
44         return node->countPrefix;
45     }
46
47     void erase(string word) {
48         TrieNode* node = root;
49         for (char c : word) {
50             int idx = c - 'a';
51             if (!node->children[idx]) return; // not present
52             node = node->children[idx];
53             node->countPrefix--;
54         }
55         node->countEnd--;
56     }
57 };
58
59 int main() {
60     TrieII trie;
61     trie.insert("apple");
62     trie.insert("apple");
63     trie.insert("app");
64
65     cout << trie.countWordsEqualTo("apple") << endl; // 2

```

```

66     cout << trie.countWordsStartingWith("app") << endl;           // 3
67
68     trie.erase("apple");
69     cout << trie.countWordsEqualTo("apple") << endl;             // 1
70     cout << trie.countWordsStartingWith("app") << endl;         // 2
71     return 0;
72 }

```

Complexity Analysis

Time Complexity. $O(L)$ for every operation. **Space Complexity.** $O(\Sigma \cdot N)$, as in Section 1.2, with two integer counters added per node.

Key Takeaway

Whenever a trie problem needs counts rather than a single boolean, annotate every node with the counters the queries actually need – typically “how many words pass through here” and/or “how many words end exactly here” – and update them incrementally on insert and erase. This is a direct generalization of the single *isEnd* flag from Section 1.2.

1.4 Longest String with All Prefixes

Problem Statement. Given an array of words, find the **longest** word such that **every prefix** of that word (of every length from 1 up to its full length) is also present, as a complete word, somewhere in the array. If there are ties in length, return the lexicographically smallest such word. If no word qualifies (not even single-character words), return the empty string.

Pattern Recognition

“*Every prefix of this word must also be a complete word in the collection*” is the keyword that distinguishes this from Section 1.2’s plain *startsWith* check: we are not asking whether a path exists, but whether **every node along that entire path** happens to be an end-of-word marker, not just the final node.

Understanding the problem carefully

Insert every word into a trie, exactly as in Section 1.2, marking end-of-word at each word’s final node. Then, for each word, walk its path through the trie and check *every* node visited (not just the last one) for the end-of-word marker. A word qualifies if and only if this check passes at every single position along its path – meaning the 1-character prefix, the 2-character prefix, and so on, are all themselves complete words that were inserted into the trie.

Check Every Node Along the Path, Not Just the Last

For each candidate word w , walk the trie character by character. At *every* node visited (after consuming each character), check that node’s end-of-word flag. If any node along the way fails this check, w does not qualify – stop checking this word immediately. If the entire walk completes with every node passing the check, w qualifies; compare it against the best answer found so far (preferring longer length, and lexicographically smaller on ties).

Why checking the whole path (not just the destination) is the correct test

The problem’s requirement is explicitly about *every prefix*, not just the word itself – a word might be present in the array (so its final node is marked end-of-word) while one of its shorter

prefixes was *never* separately inserted as its own word, which would still disqualify it. Since every node on a word's path through the trie corresponds to exactly one specific prefix of that word, checking the end-of-word flag at every node along the path is precisely equivalent to checking, for every prefix length, whether that prefix was itself inserted as a complete word.

Algorithm

1. Insert every word into a trie (as in Section 1.2).
2. For each word w (processed, say, in sorted order so that ties are resolved automatically by encountering the lexicographically smaller one first):
 - (a) Walk w 's path; at each node, check the end-of-word flag; if any check fails, discard w .
 - (b) If w survives the full walk and is longer than the current best answer (or ties in length but is lexicographically smaller, naturally handled by sorted-order processing), update the best answer.
3. Return the best answer found (or empty string if none qualified).

Dry run

Dry Run. Take $words = ["a", "banana", "app", "appl", "ap", "apply", "apple"]$.
 Sorted: "a", "ap", "app", "appl", "apple", "apply", "banana".

- "a": path is just the node for a; it is marked end-of-word (since "a" itself was inserted). Qualifies, length 1. Best so far: "a".
- "ap": nodes for a, ap; both marked (since "a" and "ap" were both inserted). Qualifies, length 2. Best so far: "ap".
- "app": nodes for a, ap, app all marked. Qualifies, length 3. Best so far: "app".
- "appl": all four prefix nodes marked. Qualifies, length 4. Best so far: "appl".
- "apple": all five prefix nodes marked (every prefix from "a" to "apple" was inserted). Qualifies, length 5. Best so far: "apple".
- "apply": prefix "appl" is marked, but at the final node ("apply"), it is marked too – but wait, we need *every* prefix; "apply" shares "a", "ap", "app", "appl" with "apple" (all marked), and "apply" itself was inserted, so it also qualifies at length 5 – tied with "apple". Since "apple" was already recorded and is lexicographically smaller than "apply", we keep "apple".
- "banana": the node for b is not marked end-of-word as a 1-letter prefix unless "b" itself was inserted – it was not. Disqualified immediately.

Final answer: "apple".

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TrieNode {
5     TrieNode* children[26] = {nullptr};
6     bool isEnd = false;

```

```

7 };
8
9 class Trie {
10 public:
11     TrieNode* root = new TrieNode();
12
13     void insert(string& word) {
14         TrieNode* node = root;
15         for (char c : word) {
16             int idx = c - 'a';
17             if (!node->children[idx]) node->children[idx] = new
18                 TrieNode();
19             node = node->children[idx];
20         }
21         node->isEnd = true;
22     }
23
24     bool allPrefixesExist(string& word) {
25         TrieNode* node = root;
26         for (char c : word) {
27             int idx = c - 'a';
28             if (!node->children[idx] || !node->children[idx]->isEnd) {
29                 return false;
30             }
31             node = node->children[idx];
32         }
33         return true;
34     }
35 };
36
37 string longestCompleteString(vector<string>& words) {
38     Trie trie;
39     for (string& w : words) trie.insert(w);
40
41     sort(words.begin(), words.end());
42
43     string best = "";
44     for (string& w : words) {
45         if (trie.allPrefixesExist(w) && w.size() > best.size()) {
46             best = w;
47         }
48     }
49     return best;
50 }
51
52 int main() {
53     vector<string> words = {"a", "banana", "app", "appl", "ap", "apply", "
54         apple"};
55     cout << longestCompleteString(words) << endl;
56     return 0;
57 }

```

Complexity Analysis

Time Complexity. Inserting all words costs $O(N)$, where N is the total number of characters across all words. Sorting costs $O(N \log n)$ (comparing n words). Checking every

word's path costs another $O(N)$ total. Overall:

$$O(N \log n).$$

Space Complexity. $O(\Sigma \cdot N)$ for the trie.

Key Takeaway

Whenever a problem's condition is phrased as “true for every prefix,” the trie check needs to inspect every node along a path, not just the final destination – a small but easy-to-miss generalization of plain *search* from Section 1.2.

1.5 Number of Distinct Substrings in a String

Problem Statement. Given a string s of length n , count the number of **distinct** (non-empty) substrings of s .

Pattern Recognition

“Count distinct substrings” reframes naturally once you notice that every substring of s is, by definition, a **prefix of some suffix** of s . Inserting *every suffix* of s into a trie therefore inserts every substring of s as some path from the root – and because a trie physically **shares** identical prefixes as the same nodes, the number of **distinct** substrings is exactly the number of **distinct nodes** the trie ends up containing.

Understanding the problem carefully

Consider inserting all n suffixes of s (the suffix starting at index 0, the suffix starting at index 1, ..., the suffix starting at index $n - 1$) into a trie, one character at a time, exactly as in Section 1.2. Every prefix of every suffix is some substring of s , and every substring of s is the prefix of at least one suffix (specifically, the suffix that starts where the substring starts) – so this process inserts every substring of s , with duplicates naturally *not* creating new nodes, since inserting a character that already has a corresponding child simply reuses that existing node.

Count New Nodes Created, Not Total Insertions

For each suffix of s (there are n of them, of lengths $n, n - 1, \dots, 1$), walk it through the trie character by character. Whenever a required child does not yet exist, create it *and increment a running counter of distinct substrings* (since a brand-new node means this exact substring – the path from root to this new node – has never been encountered before). When a required child already exists, simply move into it without incrementing the counter (this exact substring was already counted during an earlier suffix's insertion).

Why counting newly created nodes exactly counts distinct substrings

Each trie node corresponds to exactly one specific substring – the concatenation of characters along the path from the root to that node – and two different nodes can never correspond to the same substring (a trie's structure forbids two different paths spelling out the same string, since shared prefixes are always merged into the same nodes). Conversely, every substring of s does correspond to *some* node, because it is a prefix of at least one inserted suffix. So there is a perfect one-to-one correspondence between distinct substrings of s and nodes of the trie (excluding the root, which represents the empty string) – counting nodes created during

insertion is therefore counting distinct substrings directly, with no separate deduplication step needed.

Algorithm

1. Initialize an empty trie and $count \leftarrow 0$.
2. For each starting index i from 0 to $n - 1$ (each suffix $s[i..n - 1]$):
 - (a) Walk the suffix's characters from the trie's root. For each character: if the required child does not exist, create it and increment $count$; move into the child.
3. Return $count$.

Dry run

Dry Run. Take $s = \text{"ab"}$. Suffixes: "ab" , "b" .

- Insert "ab" : a does not exist at the root – create it, $count = 1$. Move into a . b does not exist there – create it, $count = 2$.
- Insert "b" : b does not exist at the root (this is a different node than the b created above, which was a child of a , not of the root) – create it, $count = 3$.

Total distinct substrings: 3 – namely "a" , "ab" , "b" , matching the direct enumeration of all substrings of "ab" .

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TrieNode {
5      TrieNode* children[26] = {nullptr};
6  };
7
8  int countDistinctSubstrings(string s) {
9      TrieNode* root = new TrieNode();
10     int n = s.size();
11     int count = 0;
12
13     for (int i = 0; i < n; i++) {
14         TrieNode* node = root;
15         for (int j = i; j < n; j++) {
16             int idx = s[j] - 'a';
17             if (!node->children[idx]) {
18                 node->children[idx] = new TrieNode();
19                 count++;
20             }
21             node = node->children[idx];
22         }
23     }
24     return count;
25 }
26
27 int main() {
28     cout << countDistinctSubstrings("ab") << endl;    // 3

```

```

29     cout << countDistinctSubstrings("aaa") << endl;    // 3 (a, aa, aaa)
30     return 0;
31 }

```

Complexity Analysis

Time Complexity. Inserting all n suffixes, of total length $n + (n - 1) + \dots + 1$, gives

$$O(n^2)$$

character insertions in the worst case, each $O(1)$ amortized trie work. (A suffix automaton or suffix array based approach can solve the same problem in $O(n \log n)$ or $O(n)$, but the trie-of-all-suffixes approach shown here is the direct, easy-to-derive $O(n^2)$ solution, appropriate for moderate string lengths.) **Space Complexity.** $O(n^2)$ in the worst case (one trie node per distinct substring, and a string can have up to $O(n^2)$ distinct substrings).

Key Takeaway

“Count distinct substrings” is solved by recognizing that a trie’s node-sharing behavior *is* a deduplication mechanism: insert everything that could possibly create a duplicate (here, every suffix, since every substring is a prefix of some suffix), and count only the genuinely new nodes created, never the total insertion attempts.

1.6 Maximum XOR of Two Numbers in an Array

Problem Statement. Given an array *nums* of non-negative integers, find the maximum value of $nums[i] \oplus nums[j]$ over all pairs $i \neq j$ (where $\oplus$ denotes bitwise XOR).

Pattern Recognition

“*Maximum XOR*” between pairs of numbers is the keyword for a **binary trie**: treat each number as a fixed-length sequence of bits (most significant bit first) and insert it into a trie exactly as Section 1.2 inserts characters of a string – except the “alphabet” here has only two symbols, 0 and 1. Once every number is in the trie, maximizing the XOR with a given number becomes a greedy walk: at each bit position, XOR is maximized by making that bit **differ**, so we always prefer the branch *opposite* to the current number’s bit, whenever it exists.

Understanding the problem carefully

XOR-ing two bits produces 1 exactly when they differ. To make the overall XOR as large as possible, we want to maximize the *most significant* differing bit first (since higher bit positions dominate the numeric value), then the next, and so on – exactly the same most-significant-first greedy logic used throughout binary search’s “binary search on the answer” problems, but here applied bit by bit through a trie rather than narrowing a numeric range.

Insert All Numbers as Bit-Paths, Then Greedily Walk for the Opposite Bit

Insert every number into a binary trie, one bit at a time from the most significant bit downward (using a fixed bit width, e.g. 32, so all numbers have aligned representations). For each number x in the array, walk the trie trying, at each bit position, to move into the child representing the *opposite* of x ’s bit at that position; if that child does not exist, settle for the only available child (the same bit). Accumulate the resulting XOR value bit by bit as you walk, and track the maximum such value found over all numbers x .

Why the greedy opposite-bit choice is always optimal

Because bit positions contribute to the final value with strictly decreasing weight (the most significant bit contributes more than the sum of all lower bits combined, for a fixed bit width), it is always better to secure a 1 at a higher bit position than to optimize any combination of lower bits – so greedily taking the opposite-bit branch whenever it is available, at every level from the most significant bit down, is guaranteed to produce the maximum possible XOR with the existing numbers in the trie, by the same most-significant-digit-first reasoning used whenever comparing fixed-width binary numbers.

Algorithm

1. Insert every number in *nums* into a binary trie, bit by bit from the most significant bit (say bit 31) to the least significant (bit 0).
2. For each number *x* in *nums*:
 - (a) Walk the trie from the root. At each bit position *b* (from most to least significant): let *desired* be the opposite of *x*'s bit *b*. If the trie node has a child for *desired*, move into it and set this bit of the running XOR result to 1; otherwise, move into the only available child and set this bit to 0.
 - (b) Update the global maximum with this number's resulting XOR value.
3. Return the global maximum.

Dry run

Dry Run. Take *nums* = [3, 10, 5, 25, 2, 8], using 5-bit representations for clarity (3=00011, 10=01010, 5=00101, 25=11001, 2=00010, 8=01000). Query the best XOR partner for 5=00101.

- Bit 4 (value 16) of 5 is 0; desired opposite is 1. Only 25 has bit 4 = 1. Move there; XOR bit 4 = 1.
- Bit 3 (value 8) of 5 is 0; desired 1. Within this branch, only 25 remains, and its bit 3 = 1 – matches desired. Move there; XOR bit 3 = 1.
- Bit 2 (value 4) of 5 is 1; desired 0. 25's bit 2 = 0 – matches. Move there; XOR bit 2 = 1.
- Bit 1 (value 2) of 5 is 0; desired 1. Only 25's branch remains here, and its bit 1 = 0 – not the desired branch, but it's the only one available. Move there; XOR bit 1 = 0.
- Bit 0 (value 1) of 5 is 1; desired 0. Only 25's bit 0 = 1 available. Move there; XOR bit 0 = 0.

Resulting XOR bits: 11100₂ = 28. Indeed $5 \oplus 25 = 28$, the maximum pairwise XOR for this array.

C++ implementation

```

1 #include <bits/stdc++.h>
2 using namespace std;
3
4 struct TrieNode {
5     TrieNode* children[2] = {nullptr, nullptr};
6 };
7
8 class BinaryTrie {
```

```

9     TrieNode* root = new TrieNode();
10     static const int BITS = 32;
11
12 public:
13     void insert(int num) {
14         TrieNode* node = root;
15         for (int b = BITS - 1; b >= 0; b--) {
16             int bit = (num >> b) & 1;
17             if (!node->children[bit]) node->children[bit] = new
18                 TrieNode();
19             node = node->children[bit];
20         }
21
22     int maxXorWith(int num) {
23         TrieNode* node = root;
24         int result = 0;
25         for (int b = BITS - 1; b >= 0; b--) {
26             int bit = (num >> b) & 1;
27             int desired = 1 - bit;
28             if (node->children[desired]) {
29                 result |= (1 << b);
30                 node = node->children[desired];
31             } else {
32                 node = node->children[bit];
33             }
34         }
35         return result;
36     }
37 };
38
39 int findMaximumXOR(vector<int>& nums) {
40     BinaryTrie trie;
41     for (int x : nums) trie.insert(x);
42
43     int maxXor = 0;
44     for (int x : nums) {
45         maxXor = max(maxXor, trie.maxXorWith(x));
46     }
47     return maxXor;
48 }
49
50 int main() {
51     vector<int> nums = {3, 10, 5, 25, 2, 8};
52     cout << "Maximum XOR: " << findMaximumXOR(nums) << endl;
53     return 0;
54 }

```

Complexity Analysis

Time Complexity. Inserting all n numbers costs $O(n \cdot BITS)$. Querying the best partner for each of the n numbers costs another $O(n \cdot BITS)$. Overall:

$$O(n \cdot BITS) = O(n),$$

treating the fixed bit width (32) as a constant – a dramatic improvement over the $O(n^2)$ brute-force check of every pair. **Space Complexity.** $O(n \cdot BITS)$ for the trie in the worst case.

Key Takeaway

A binary trie is a character trie with a two-letter alphabet – every technique from Sections 1.2–1.5 carries over unchanged, just with bits instead of letters. Whenever a problem involves maximizing or counting based on XOR (or, more generally, bitwise structure) across many numbers, a binary trie processed most-significant-bit-first is almost always the intended technique.

1.7 Maximum XOR With an Element from Array (Queries)

Problem Statement. Given an array $nums$ and a list of queries, where each query is a pair (x_i, m_i) , answer for each query the maximum value of $x_i \oplus nums[j]$ over all j with $nums[j] \leq m_i$. If no element of $nums$ satisfies $nums[j] \leq m_i$, the answer for that query is -1 .

Pattern Recognition

This is Section 1.6’s binary trie with one new constraint layered on top: each query only considers a **subset** of $nums$ (those $\leq m_i$), and the subset differs per query. The keyword combination “*maximum XOR*” plus “*subject to a threshold that varies per query*” signals an **offline** technique: sort the queries by their threshold and process them in increasing order, incrementally growing the trie to include exactly the elements eligible for each query, so that no element is ever inserted before it is allowed to be, and no element needs to be removed once inserted.

Understanding the problem carefully

If queries arrived with arbitrary, unsorted thresholds m_i , we would need to repeatedly figure out “which elements of $nums$ are $\leq m_i$ ” for each query independently. But if we instead **sort the queries by m_i** and process them in increasing order, the set of eligible elements only ever **grows** from one query to the next (a higher threshold can only admit more elements, never fewer) – so we can maintain a single pointer into a separately sorted copy of $nums$, inserting new elements into the trie as the threshold grows, and never needing to remove anything.

Offline: Sort Both Queries and the Array by Threshold

Sort $nums$ in increasing order. Sort the queries by m_i , remembering each query’s original index (to place answers back correctly). Maintain a pointer j into the sorted $nums$, starting at 0. For each query, in increasing order of m_i : while j has not exhausted $nums$ and $nums[j] \leq m_i$, insert $nums[j]$ into the binary trie and advance j . Then, if the trie is non-empty, answer this query using Section 1.6’s $maxXorWith(x_i)$; otherwise (no eligible elements at all), answer -1 .

Why processing queries in sorted threshold order is correct and never wastes work

Because eligibility only grows as the threshold increases, every element inserted into the trie while answering an earlier (smaller-threshold) query remains correctly eligible for every later (larger-threshold) query too – nothing inserted ever needs to be removed. This means the pointer j only ever moves forward, each element of $nums$ is inserted into the trie exactly once across the entire batch of queries, and the total insertion work across all queries combined is $O(n)$ insertions, not $O(nq)$ for q queries – turning what looks like a per-query rebuild into a single shared incremental structure.

Algorithm

1. Sort *nums* in increasing order.
2. Sort the queries by m_i , keeping track of each query's original position.
3. Initialize an empty binary trie and a pointer $j \leftarrow 0$.
4. For each query (x_i, m_i) , in sorted order:
 - (a) While $j < n$ and $nums[j] \leq m_i$: insert $nums[j]$ into the trie; $j \leftarrow j + 1$.
 - (b) If the trie is empty: this query's answer is -1 .
 - (c) Else: this query's answer is $maxXorWith(x_i)$.
5. Place each computed answer back at its query's original position; return the answers.

Dry run

Dry Run. Take $nums = [0, 1, 2, 3, 4]$ and queries $[(3, 1), (1, 3), (5, 6)]$.

Sorted $nums$ unchanged: $[0, 1, 2, 3, 4]$. Sorted queries by m_i : $(3, 1), (1, 3), (5, 6)$ (already in this order).

- Query $(x=3, m=1)$: insert $nums[j]$ while ≤ 1 : insert 0 ($j \rightarrow 1$), insert 1 ($j \rightarrow 2$); next is $2 > 1$, stop. Trie has $\{0, 1\}$. $maxXorWith(3)$: $3 \oplus 0 = 3$, $3 \oplus 1 = 2$; best is 3.
- Query $(x=1, m=3)$: insert while ≤ 3 : insert 2 ($j \rightarrow 3$), insert 3 ($j \rightarrow 4$); next is $4 > 3$, stop. Trie has $\{0, 1, 2, 3\}$. $maxXorWith(1)$: $1 \oplus 0 = 1$, $1 \oplus 1 = 0$, $1 \oplus 2 = 3$, $1 \oplus 3 = 2$; best is 3.
- Query $(x=5, m=6)$: insert while ≤ 6 : insert 4 ($j \rightarrow 5$); $j = n$, stop. Trie has $\{0, 1, 2, 3, 4\}$. $maxXorWith(5)$: best pairing is $5 \oplus 2 = 7$ (checking all: $5=101$, $0=000 \rightarrow 101=5$, $1=001 \rightarrow 100=4$, $2=010 \rightarrow 111=7$, $3=011 \rightarrow 110=6$, $4=100 \rightarrow 001=1$); best is 7.

Answers in original query order: $[3, 3, 7]$.

C++ implementation

```

1  #include <bits/stdc++.h>
2  using namespace std;
3
4  struct TrieNode {
5      TrieNode* children[2] = {nullptr, nullptr};
6  };
7
8  class BinaryTrie {
9      TrieNode* root = new TrieNode();
10     static const int BITS = 32;
11     bool empty = true;
12
13 public:
14     void insert(int num) {
15         empty = false;
16         TrieNode* node = root;
17         for (int b = BITS - 1; b >= 0; b--) {
18             int bit = (num >> b) & 1;
19             if (!node->children[bit]) node->children[bit] = new
20                 TrieNode();
21             node = node->children[bit];

```

```

21     }
22 }
23
24 bool isEmpty() { return empty; }
25
26 int maxXorWith(int num) {
27     TrieNode* node = root;
28     int result = 0;
29     for (int b = BITS - 1; b >= 0; b--) {
30         int bit = (num >> b) & 1;
31         int desired = 1 - bit;
32         if (node->children[desired]) {
33             result |= (1 << b);
34             node = node->children[desired];
35         } else {
36             node = node->children[bit];
37         }
38     }
39     return result;
40 }
41 };
42
43 vector<int> maximizeXor(vector<int>& nums, vector<vector<int>>& queries
44 ) {
45     sort(nums.begin(), nums.end());
46
47     int q = queries.size();
48     vector<int> order(q);
49     for (int i = 0; i < q; i++) order[i] = i;
50     sort(order.begin(), order.end(), [&](int a, int b) {
51         return queries[a][1] < queries[b][1];
52     });
53
54     BinaryTrie trie;
55     vector<int> answers(q);
56     int j = 0, n = nums.size();
57
58     for (int idx : order) {
59         int x = queries[idx][0], m = queries[idx][1];
60         while (j < n && nums[j] <= m) {
61             trie.insert(nums[j]);
62             j++;
63         }
64         answers[idx] = trie.isEmpty() ? -1 : trie.maxXorWith(x);
65     }
66     return answers;
67 }
68
69 int main() {
70     vector<int> nums = {0, 1, 2, 3, 4};
71     vector<vector<int>> queries = {{3,1}, {1,3}, {5,6}};
72
73     for (int ans : maximizeXor(nums, queries)) cout << ans << " ";
74     cout << endl;
75     return 0;
76 }

```

Complexity Analysis

Time Complexity. Sorting *nums* and the queries costs $O(n \log n + q \log q)$. Each element of *nums* is inserted into the trie exactly once across the whole batch, costing $O(n \cdot BITS)$ total. Each query performs one $O(BITS)$ trie walk. Overall:

$$O((n + q) \log(n + q) + (n + q) \cdot BITS).$$

Space Complexity. $O(n \cdot BITS)$ for the trie, plus $O(q)$ for the answers and sort order.

Key Takeaway

This closing problem of the chapter – and of this entire multi-chapter project – is a deliberate capstone combining two ideas: the binary trie from Section 1.6, and the classic **offline query** technique of sorting queries by their constraint and processing them alongside a sorted version of the data, so that a single incrementally-built structure serves every query without ever needing to be rebuilt or have elements removed. Whenever queries each restrict attention to a growing prefix of sorted data, check whether sorting the queries themselves by that same threshold turns many independent queries into one shared incremental pass.